VulnGuard API — Complete Code Listing

COM661 Full Stack Strategies and Development

CW1 — Individual Full Stack Application Development (Back End)

Student: Hritik Kumar Sah

Student Number: B00923557

Table of Contents

1. app.py — Application Factory & Development Server
2. config.py — Configuration Classes
3. routes/__init__.py — Blueprint Registration
4. routes/auth/__init__.py — Auth Blueprint
5. routes/auth/routes.py — Auth Routes
6. routes/auth/validation.py — Auth Validation
7. routes/vulnerabilities/__init__.py — Vulnerabilities Blueprint
8. routes/vulnerabilities/routes.py — Vulnerability Routes
9. routes/vulnerabilities/validation.py — Vulnerability Validation
10. routes/remediation/__init__.py — Remediation Blueprint
11. routes/remediation/routes.py — Remediation Routes
12. routes/remediation/validation.py — Remediation Validation
13. routes/activity_log/__init__.py — Activity Log Blueprint
14. routes/activity_log/routes.py — Activity Log Routes
15. routes/activity_log/validation.py — Activity Log Validation
16. routes/analytics/__init__.py — Analytics Blueprint
17. routes/analytics/routes.py — Analytics Routes
18. routes/admin/__init__.py — Admin Blueprint
19. routes/admin/routes.py — Admin Routes
20. middleware/auth_middleware.py — JWT Authentication
21. middleware/rbac_middleware.py — Role-Based Access Control
22. middleware/error_handler.py — Centralised Error Handling
23. utils/helpers.py — Utility Helpers
24. utils/validators.py — Shared Validators
25. seeds/seed_data.py — Database Seeder

---

1. app.py

File: `backend/app.py`

```
"""VulnGuard — Flask application factory and development server entry
point.

Usage:
    python app.py                  # Start development server
    python app.py --port 8000      # Use custom port
    python app.py --no-debug       # Disable debug / auto-reload
"""

import argparse
import os
import logging
from logging.handlers import RotatingFileHandler
from flask import Flask, jsonify, request, g
from flask_cors import CORS
```

```
from pymongo import MongoClient
from pymongo.errors import ConnectionFailure, ServerSelectionTimeoutError
import time

# Optional: load .env file if python-dotenv is installed
try:
    from dotenv import load_dotenv
    load_dotenv(os.path.join(os.path.dirname(__file__), '.env'))
except ImportError:
    pass  # python-dotenv not installed — fall back to OS environment
variables

from config import config_by_name


def _configure_logging(app: Flask) -> None:
    """Set up structured logging with console and file handlers."""
    log_level = logging.DEBUG if app.debug else logging.INFO
    log_format = logging.Formatter(
        '[%(asctime)s] %(levelname)s in %(module)s (%(funcName)s:%
(lineno)d): %(message)s',
        datefmt='%Y-%m-%d %H:%M:%S'
    )

    console_handler = logging.StreamHandler()
    console_handler.setFormatter(log_format)
    console_handler.setLevel(log_level)

    # File handler — rotates at 10 MB, keeps 5 backups
    log_dir = os.path.join(os.path.dirname(os.path.abspath(__file__)),
'logs')
    os.makedirs(log_dir, exist_ok=True)
    file_handler = RotatingFileHandler(
        os.path.join(log_dir, 'vulnguard.log'),
        maxBytes=10 * 1024 * 1024, backupCount=5,
    )
    file_handler.setFormatter(log_format)
    file_handler.setLevel(log_level)

    app.logger.handlers.clear()
    app.logger.addHandler(console_handler)
    app.logger.addHandler(file_handler)
    app.logger.setLevel(log_level)

    # Quieten noisy third-party loggers
    logging.getLogger('werkzeug').setLevel(logging.WARNING)
    logging.getLogger('pymongo').setLevel(logging.WARNING)


def create_app(config_name: str | None = None) -> Flask:
    """Create and configure the Flask application."""
    if config_name is None:
        config_name = os.environ.get('FLASK_ENV', 'development')
```

```
    app = Flask(__name__)
    app.config.from_object(config_by_name[config_name])

    _configure_logging(app)
    app.logger.info("Starting VulnGuard API [env=%s]", config_name)

    CORS(app, resources={r"/api/*": {"origins": "*"}})

    # Initialise MongoDB
    client = MongoClient(app.config['MONGO_URI'],
serverSelectionTimeoutMS=5000)
    db_name = app.config['MONGO_URI'].rsplit('/', 1)[-1].split('?')[0]
    db = client[db_name]
    app.extensions["mongo_db"] = db

    # Test MongoDB connection at startup
    try:
        client.admin.command('ping')
        app.logger.info("MongoDB connected successfully [%s]",
app.config['MONGO_URI'])
    except (ConnectionFailure, ServerSelectionTimeoutError) as e:
        app.logger.warning(
            "WARNING: MongoDB is not reachable at %s — %s. "
            "The server will start, but database operations will fail.",
            app.config['MONGO_URI'], e,
        )

    # Create indexes
    try:
        _create_indexes(db)
    except Exception as e:
        app.logger.warning("Could not create indexes at startup: %s", e)

    # Register error handlers and routes
    from middleware.error_handler import register_error_handlers
    register_error_handlers(app)

    from routes import register_blueprints
    register_blueprints(app)

    # Request timing middleware
    @app.before_request
    def _before_request():
        g.start_time = time.time()

    @app.after_request
    def _after_request(response):
        duration = round((time.time() - getattr(g, 'start_time',
time.time())) * 1000, 2)
        app.logger.info("%s %s %s %sms", request.method, request.path,
                        response.status_code, duration)
        return response

    @app.route('/api/v1/health', methods=['GET'])
```

```
    def health_check():
        """Return API health status."""
        return jsonify({"status": "success", "message": "VulnGuard API is
running"}), 200

    return app


def _create_indexes(db) -> None:
    """Create all required indexes for optimal query performance.

    Args:
        db: PyMongo database instance.
    """
    # Single field indexes
    db.vulnerabilities.create_index("severity")
    db.vulnerabilities.create_index("status")
    db.vulnerabilities.create_index("cvss_score")
    db.vulnerabilities.create_index("department")
    db.vulnerabilities.create_index("patch_due_date")
    db.vulnerabilities.create_index("patch_applied")
    db.vulnerabilities.create_index("created_at")
    db.vulnerabilities.create_index("vulnerability_type")
    db.vulnerabilities.create_index("discovery_method")

    # Compound index for common filter combinations
    db.vulnerabilities.create_index([
        ("severity", 1),
        ("status", 1),
        ("department", 1)
    ])

    # Text index for search
    db.vulnerabilities.create_index([
        ("vulnerability_title", "text"),
        ("description", "text")
    ])

    # Geospatial index for location-aware queries
    db.vulnerabilities.create_index([("location", "2dsphere")])

    # User collection indexes
    db.users.create_index("email", unique=True)
    db.users.create_index("username", unique=True)

    # Token blacklist index (TTL — auto-delete after 24 h)
    db.blacklist.create_index("token")
    db.blacklist.create_index("blacklisted_at", expireAfterSeconds=86400)


if __name__ == '__main__':
    parser = argparse.ArgumentParser(description="VulnGuard Development
Server")
    parser.add_argument('--host', default='0.0.0.0', help='Host to bind
```

```
(default: 0.0.0.0)')
    parser.add_argument('--port', type=int, default=5000, help='Port to
bind (default: 5000)')
    parser.add_argument('--no-debug', action='store_true', help='Disable
debug mode')
    args = parser.parse_args()

    app = create_app()
    print(f"\n🚀  Starting VulnGuard API on http://{args.host}:
{args.port}")
    app.run(host=args.host, port=args.port, debug=not args.no_debug)
```

---

2. config.py

File: `backend/config.py`

```
"""Application configuration classes for different environments."""

import os

# Optional: load .env file if python-dotenv is installed
try:
    from dotenv import load_dotenv
    load_dotenv()
except ImportError:
    pass  # python-dotenv not installed - fall back to OS environment
variables


class Config:
    """Base configuration."""

    SECRET_KEY: str = os.environ.get('SECRET_KEY', 'change-me-in-
production')
    MONGO_URI: str = os.environ.get('MONGO_URI',
'mongodb://localhost:27017/vulnguard')
    JWT_EXPIRY_HOURS: int = int(os.environ.get('JWT_EXPIRY_HOURS', '1'))
    ITEMS_PER_PAGE: int = int(os.environ.get('ITEMS_PER_PAGE', '10'))
    MAX_ITEMS_PER_PAGE: int = 100
    MAX_CONTENT_LENGTH: int = 16 * 1024 * 1024  # 16 MB max request size


class DevelopmentConfig(Config):
    """Development configuration."""

    DEBUG: bool = True


class TestingConfig(Config):
    """Testing configuration."""
```

```python
    TESTING: bool = True
    MONGO_URI: str = os.environ.get(
        'TEST_MONGO_URI', 'mongodb://localhost:27017/vulnguard_test'
    )


class ProductionConfig(Config):
    """Production configuration."""

    DEBUG: bool = False


config_by_name = {
    'development': DevelopmentConfig,
    'testing': TestingConfig,
    'production': ProductionConfig,
}
```

---

3. routes/__init__.py

File: backend/routes/__init__.py

```python
"""VulnGuard Blueprints Package.

Provides a single entry-point for registering all API blueprints:
auth, vulnerabilities, remediation, activity_log, analytics, admin.
"""

from flask import Flask


def register_blueprints(app: Flask) -> None:
    """Import and register all API blueprints with the application."""
    from routes.auth import auth_bp
    from routes.vulnerabilities import vulnerabilities_bp
    from routes.remediation import remediation_bp
    from routes.activity_log import activity_log_bp
    from routes.analytics import analytics_bp
    from routes.admin import admin_bp

    app.register_blueprint(auth_bp)
    app.register_blueprint(vulnerabilities_bp)
    app.register_blueprint(remediation_bp)
    app.register_blueprint(activity_log_bp)
    app.register_blueprint(analytics_bp)
    app.register_blueprint(admin_bp)

    app.logger.info("Registered 6 blueprints")
```

4. routes/auth/__init__.py

File: backend/routes/auth/__init__.py

```
"""Auth blueprint — user registration, login, profile, and token
management."""

from flask import Blueprint

auth_bp = Blueprint('auth', __name__, url_prefix='/api/v1/auth')

from routes.auth import routes  # noqa: E402, F401
```

5. routes/auth/routes.py

File: backend/routes/auth/routes.py

```
"""Authentication and user management endpoints."""

from datetime import datetime, timedelta, timezone
from uuid import uuid4
from flask import request, jsonify, current_app
from bson import ObjectId
import bcrypt
import jwt

from middleware.auth_middleware import token_required, _extract_token
from routes.auth.validation import (
    validate_registration_input,
    validate_login_input,
    validate_profile_update,
    validate_change_password,
)
from utils.helpers import get_db, serialize_doc, utcnow
from routes.auth import auth_bp


@auth_bp.route('/register', methods=['POST'])
def register():
    """Register a new user account."""
    data = request.get_json(silent=True)
    if not data:
        return jsonify({"status": "error", "message": "Request body must
be valid JSON", "code": 400}), 400

    is_valid, error_msg = validate_registration_input(data)
    if not is_valid:
        return jsonify({"status": "error", "message": error_msg, "code":
```

```
422}), 422

    db = get_db()

    # Check for duplicate email
    if db.users.find_one({"email": data['email'].lower().strip()}):
        return jsonify({"status": "error", "message": "Email already
registered", "code": 400}), 400

    # Check for duplicate username
    if db.users.find_one({"username": data['username'].strip()}):
        return jsonify({"status": "error", "message": "Username already
taken", "code": 400}), 400

    # Hash password
    password_hash = bcrypt.hashpw(
        data['password'].encode('utf-8'),
        bcrypt.gensalt(rounds=12)
    )

    now = utcnow()
    user_doc = {
        "username": data['username'].strip(),
        "email": data['email'].lower().strip(),
        "password_hash": password_hash.decode('utf-8'),
        "role": "analyst",  # Default role
        "is_active": True,
        "created_at": now,
        "last_login": None,
    }

    result = db.users.insert_one(user_doc)
    user_doc['_id'] = result.inserted_id

    # Remove password_hash from response
    response_data = serialize_doc(user_doc)
    response_data.pop('password_hash', None)

    return jsonify({
        "status": "success",
        "data": response_data,
        "message": "User registered successfully"
    }), 201


@auth_bp.route('/login', methods=['POST'])
def login():
    """Authenticate user and return JWT token."""
    data = request.get_json(silent=True)
    if not data:
        return jsonify({"status": "error", "message": "Request body must
be valid JSON", "code": 400}), 400

    is_valid, error_msg = validate_login_input(data)
```

```
    if not is_valid:
        return jsonify({"status": "error", "message": error_msg, "code":
400}), 400

    db = get_db()
    user = db.users.find_one({"email": data['email'].lower().strip()})

    if not user:
        return jsonify({"status": "error", "message": "Invalid email or
password", "code": 401}), 401

    if not user.get('is_active', True):
        return jsonify({"status": "error", "message": "Account is
deactivated. Contact an administrator.", "code": 401}), 401

    if not bcrypt.checkpw(data['password'].encode('utf-8'),
user['password_hash'].encode('utf-8')):
        return jsonify({"status": "error", "message": "Invalid email or
password", "code": 401}), 401

    # Update last login
    db.users.update_one({"_id": user['_id']}, {"$set": {"last_login":
utcnow()}})

    expiry_hours = current_app.config.get('JWT_EXPIRY_HOURS', 1)
    payload = {
        "user_id": str(user['_id']),
        "username": user['username'],
        "role": user['role'],
        "exp": datetime.now(timezone.utc) + timedelta(hours=expiry_hours),
        "iat": datetime.now(timezone.utc),
        "jti": str(uuid4()),
    }
    token = jwt.encode(payload, current_app.config['SECRET_KEY'],
algorithm="HS256")

    return jsonify({
        "status": "success",
        "data": {
            "token": token,
            "user": {
                "_id": str(user['_id']),
                "username": user['username'],
                "email": user['email'],
                "role": user['role'],
            }
        },
        "message": "Login successful"
    }), 200


@auth_bp.route('/profile', methods=['GET'])
@token_required
def get_profile(current_user: dict):
```

```
    """Get current authenticated user's profile."""
    db = get_db()
    user = db.users.find_one({"_id": ObjectId(current_user['user_id'])})

    if not user:
        return jsonify({"status": "error", "message": "User not found",
"code": 404}), 404

    user_data = serialize_doc(user)
    user_data.pop('password_hash', None)

    return jsonify({"status": "success", "data": user_data}), 200


@auth_bp.route('/profile', methods=['PUT'])
@token_required
def update_profile(current_user: dict):
    """Update current user's profile details."""
    data = request.get_json(silent=True)
    if not data:
        return jsonify({"status": "error", "message": "Request body must
be valid JSON", "code": 400}), 400

    is_valid, error_msg = validate_profile_update(data)
    if not is_valid:
        return jsonify({"status": "error", "message": error_msg, "code":
422}), 422

    db = get_db()
    update_fields = {}

    if 'username' in data:
        existing = db.users.find_one({
            "username": data['username'].strip(),
            "_id": {"$ne": ObjectId(current_user['user_id'])}
        })
        if existing:
            return jsonify({"status": "error", "message": "Username
already taken", "code": 400}), 400
        update_fields['username'] = data['username'].strip()

    if 'email' in data:
        existing = db.users.find_one({
            "email": data['email'].lower().strip(),
            "_id": {"$ne": ObjectId(current_user['user_id'])}
        })
        if existing:
            return jsonify({"status": "error", "message": "Email already
registered", "code": 400}), 400
        update_fields['email'] = data['email'].lower().strip()

    if not update_fields:
        return jsonify({"status": "error", "message": "No valid fields to
update", "code": 400}), 400
```

```
    db.users.update_one(
        {"_id": ObjectId(current_user['user_id'])},
        {"$set": update_fields}
    )

    user = db.users.find_one({"_id": ObjectId(current_user['user_id'])})
    user_data = serialize_doc(user)
    user_data.pop('password_hash', None)

    return jsonify({"status": "success", "data": user_data, "message":
"Profile updated successfully"}), 200


@auth_bp.route('/refresh', methods=['POST'])
@token_required
def refresh_token(current_user: dict):
    """Refresh JWT token before expiry."""
    expiry_hours = current_app.config.get('JWT_EXPIRY_HOURS', 1)
    payload = {
        "user_id": current_user['user_id'],
        "username": current_user['username'],
        "role": current_user['role'],
        "exp": datetime.now(timezone.utc) + timedelta(hours=expiry_hours),
        "iat": datetime.now(timezone.utc),
        "jti": str(uuid4()),
    }
    token = jwt.encode(payload, current_app.config['SECRET_KEY'],
algorithm="HS256")

    return jsonify({
        "status": "success",
        "data": {"token": token},
        "message": "Token refreshed successfully"
    }), 200


@auth_bp.route('/change-password', methods=['PUT'])
@token_required
def change_password(current_user: dict):
    """Change current user's password."""
    data = request.get_json(silent=True)
    if not data:
        return jsonify({"status": "error", "message": "Request body must
be valid JSON", "code": 400}), 400

    is_valid, error_msg = validate_change_password(data)
    if not is_valid:
        return jsonify({"status": "error", "message": error_msg, "code":
422}), 422

    db = get_db()
    user = db.users.find_one({"_id": ObjectId(current_user['user_id'])})
```

```
    if not user:
        return jsonify({"status": "error", "message": "User not found",
"code": 404}), 404

    if not bcrypt.checkpw(data['current_password'].encode('utf-8'),
user['password_hash'].encode('utf-8')):
        return jsonify({"status": "error", "message": "Current password is
incorrect", "code": 400}), 400

    new_hash = bcrypt.hashpw(
        data['new_password'].encode('utf-8'),
        bcrypt.gensalt(rounds=12)
    )

    db.users.update_one(
        {"_id": user['_id']},
        {"$set": {"password_hash": new_hash.decode('utf-8')}}
    )

    return jsonify({"status": "success", "message": "Password changed
successfully"}), 200


@auth_bp.route('/logout', methods=['POST'])
@token_required
def logout(current_user: dict):
    """Logout by adding the current token to the blacklist collection.

    Because JWTs cannot be destroyed before their expiry, the token is
    inserted into a ``blacklist`` collection.  The ``token_required``
    decorator checks this collection on every request.
    """
    token = _extract_token()

    db = get_db()

    # Avoid duplicates - only insert if not already blacklisted
    if not db.blacklist.find_one({"token": token}):
        db.blacklist.insert_one({
            "token": token,
            "user_id": current_user['user_id'],
            "blacklisted_at": utcnow(),
        })

    return jsonify({"status": "success", "message": "Logged out
successfully"}), 200
```

---

6. routes/auth/validation.py

File: `backend/routes/auth/validation.py`

```python
"""Input validation for authentication endpoints."""

from utils.validators import (
    validate_string, validate_email, validate_password
)


def validate_registration_input(data: dict) -> tuple[bool, str]:
    """Validate user registration input (username, email, password)."""
    errors = []
    errors.extend(validate_string(data.get('username'), 'username',
min_len=3, max_len=50))
    errors.extend(validate_email(data.get('email', '')))
    errors.extend(validate_password(data.get('password', '')))
    return (False, "; ".join(errors)) if errors else (True, "")


def validate_login_input(data: dict) -> tuple[bool, str]:
    """Validate login input (email, password required)."""
    errors = []
    if not data.get('email'):
        errors.append("'email' is required")
    if not data.get('password'):
        errors.append("'password' is required")
    return (False, "; ".join(errors)) if errors else (True, "")


# Additional validators: validate_profile_update(),
validate_change_password()
# All follow the same pattern: collect errors, return (bool, error_string)
tuple
```

---

7. routes/vulnerabilities/__init__.py

File: `backend/routes/vulnerabilities/__init__.py`

```python
"""Vulnerabilities blueprint — CRUD operations for vulnerability
records."""

from flask import Blueprint

vulnerabilities_bp = Blueprint('vulnerabilities', __name__,
url_prefix='/api/v1/vulnerabilities')

from routes.vulnerabilities import routes  # noqa: E402, F401
```

---

8. routes/vulnerabilities/routes.py

File: backend/routes/vulnerabilities/routes.py

```python
"""Vulnerability CRUD endpoints."""

import math
from flask import request, jsonify
from bson import ObjectId

from middleware.auth_middleware import token_required
from middleware.rbac_middleware import role_required
from routes.vulnerabilities.validation import validate_vulnerability_input
from utils.helpers import (
    get_db, validate_object_id, serialize_doc, get_pagination_params,
    utcnow, calculate_risk_score,
)
from utils.validators import parse_iso_date
from routes.vulnerabilities import vulnerabilities_bp


@vulnerabilities_bp.route('/', methods=['GET'])
def get_vulnerabilities():
    """Retrieve paginated list of vulnerabilities with optional filters.

    Supports:
    - Exact match: ``severity``, ``status``, ``asset_type``,
``department``, ``assigned_to``
    - Multi-value ($in):  ``severity_in``, ``status_in`` (comma-separated)
    - Range ($gte/$lte):  ``min_cvss``, ``max_cvss``
    - Regex search:       ``title_regex``
    - Full-text search:   ``search``
    - Boolean:            ``patch_applied``
    - $or combinators:    ``or_severity``, ``or_status``, ``or_min_cvss``,
``or_department``
    - Cursor-based pagination: ``after`` (ObjectId of last item)
    """
    db = get_db()
    args = request.args

    page, per_page = get_pagination_params(args)

    # Build query filter
    query = {}

    # --- Exact-match filters ---
    if args.get('severity'):
        query['severity'] = args['severity']
    if args.get('status'):
        query['status'] = args['status']
    if args.get('asset_type'):
        query['asset_type'] = args['asset_type']
    if args.get('department'):
        query['department'] = args['department']
    if args.get('assigned_to'):
```

```
        query['assigned_to'] = args['assigned_to']
    if args.get('patch_applied') is not None and args.get('patch_applied')
!= '':
        query['patch_applied'] = args['patch_applied'].lower() == 'true'

    # --- Multi-value ($in) filters ---
    if args.get('severity_in'):
        query['severity'] = {"$in": [s.strip() for s in
args['severity_in'].split(',') if s.strip()]}
    if args.get('status_in'):
        query['status'] = {"$in": [s.strip() for s in
args['status_in'].split(',') if s.strip()]}

    # --- Regex filter on vulnerability_title ---
    if args.get('title_regex'):
        query['vulnerability_title'] = {"$regex": args['title_regex'],
"$options": "i"}

    # --- CVSS range filters ---
    if args.get('min_cvss') or args.get('max_cvss'):
        cvss_filter = {}
        if args.get('min_cvss'):
            try:
                cvss_filter['$gte'] = float(args['min_cvss'])
            except ValueError:
                pass
        if args.get('max_cvss'):
            try:
                cvss_filter['$lte'] = float(args['max_cvss'])
            except ValueError:
                pass
        if cvss_filter:
            query['cvss_score'] = cvss_filter

    # --- Full-text search ---
    if args.get('search'):
        query['$text'] = {'$search': args['search']}

    # --- $or filter (combine multiple conditions) ---
    # e.g. ?or_severity=Critical&or_status=Open  → matches
severity=Critical OR status=Open
    or_clauses = []
    if args.get('or_severity'):
        or_clauses.append({"severity": args['or_severity']})
    if args.get('or_status'):
        or_clauses.append({"status": args['or_status']})
    if args.get('or_min_cvss'):
        try:
            or_clauses.append({"cvss_score": {"$gte":
float(args['or_min_cvss'])}})
        except ValueError:
            pass
    if args.get('or_department'):
        or_clauses.append({"department": args['or_department']})
```

```
    if or_clauses:
        query['$or'] = or_clauses

    # Sorting
    sort_by = args.get('sort_by', 'created_at')
    sort_order_str = args.get('sort_order', 'desc')
    sort_order = -1 if sort_order_str == 'desc' else 1

    valid_sort_fields = ['cvss_score', 'patch_due_date', 'created_at',
'severity', 'status', 'vulnerability_title']
    if sort_by not in valid_sort_fields:
        sort_by = 'created_at'

    # --- Cursor-based (keyset) pagination ---
    after = args.get('after')
    if after and validate_object_id(after):
        if sort_order == -1:
            query['_id'] = {"$lt": ObjectId(after)}
        else:
            query['_id'] = {"$gt": ObjectId(after)}

        cursor = db.vulnerabilities.find(query).sort("_id",
sort_order).limit(per_page)
        vulnerabilities = [serialize_doc(doc) for doc in cursor]
        total_count = db.vulnerabilities.count_documents(
            {k: v for k, v in query.items() if k != '_id'}
        )

        return jsonify({
            "status": "success",
            "data": vulnerabilities,
            "pagination": {
                "per_page": per_page,
                "total": total_count,
                "next_after": vulnerabilities[-1]['_id'] if
vulnerabilities else None,
            }
        }), 200

    # --- Standard skip/limit pagination ---
    total_count = db.vulnerabilities.count_documents(query)
    total_pages = math.ceil(total_count / per_page) if total_count > 0
else 1

    skip = (page - 1) * per_page
    cursor = db.vulnerabilities.find(query).sort(sort_by,
sort_order).skip(skip).limit(per_page)

    vulnerabilities = [serialize_doc(doc) for doc in cursor]

    return jsonify({
        "status": "success",
        "data": vulnerabilities,
        "pagination": {
```

```
            "page": page,
            "per_page": per_page,
            "total": total_count,
            "pages": total_pages,
        }
    }), 200


@vulnerabilities_bp.route('/<vuln_id>', methods=['GET'])
def get_vulnerability(vuln_id: str):
    """Retrieve a single vulnerability by ID."""
    if not validate_object_id(vuln_id):
        return jsonify({"status": "error", "message": "Invalid ObjectId
format", "code": 422}), 422

    db = get_db()
    vuln = db.vulnerabilities.find_one({"_id": ObjectId(vuln_id)})

    if not vuln:
        return jsonify({"status": "error", "message": "Vulnerability not
found", "code": 404}), 404

    return jsonify({"status": "success", "data": serialize_doc(vuln)}),
200


@vulnerabilities_bp.route('/', methods=['POST'])
@token_required
@role_required('admin', 'analyst')
def create_vulnerability(current_user: dict):
    """Create a new vulnerability record."""
    data = request.get_json(silent=True)
    if not data:
        return jsonify({"status": "error", "message": "Request body must
be valid JSON", "code": 400}), 400

    is_valid, error_msg = validate_vulnerability_input(data,
is_update=False)
    if not is_valid:
        return jsonify({"status": "error", "message": error_msg, "code":
422}), 422

    now = utcnow()

    vuln_doc = {
        "vulnerability_title": data['vulnerability_title'].strip(),
        "description": data['description'].strip(),
        "cve_id": data.get('cve_id', '').strip() or None,
        "severity": data['severity'],
        "status": data['status'],
        "cvss_score": float(data['cvss_score']),
        "asset_name": data['asset_name'].strip(),
        "asset_type": data['asset_type'],
        "vulnerability_type": data['vulnerability_type'],
```

```
        "discovery_method": data['discovery_method'],
        "department": data['department'].strip(),
        "affected_versions": data.get('affected_versions', []),
        "attack_vector": data.get('attack_vector'),
        "exploitability": data.get('exploitability'),
        "patch_due_date": parse_iso_date(data.get('patch_due_date')),
        "patch_applied": data.get('patch_applied', False),
        "assigned_to": data.get('assigned_to', '').strip() or None,
        "reported_by": data['reported_by'].strip(),
        "risk_score": calculate_risk_score(
            float(data['cvss_score']),
            data.get('exploitability'),
            data['severity'],
        ),
        "remediation_steps": [],
        "activity_log": [],
        "location": _build_geojson(data.get('location')),
        "created_at": now,
        "updated_at": now,
        "created_by": current_user['username'],
    }

    db = get_db()
    result = db.vulnerabilities.insert_one(vuln_doc)
    vuln_doc['_id'] = result.inserted_id

    return jsonify({
        "status": "success",
        "data": serialize_doc(vuln_doc),
        "message": "Vulnerability created successfully"
    }), 201


@vulnerabilities_bp.route('/bulk', methods=['POST'])
@token_required
@role_required('admin', 'analyst')
def bulk_create_vulnerabilities(current_user: dict):
    """Create multiple vulnerability records in a single request using
insert_many.

    Expects a JSON body with a ``vulnerabilities`` array (2–50 items).
    Each item is validated individually.  If any item fails validation the
    entire batch is rejected with a 422 listing per-item errors.

    Returns 201 with the list of created documents.
    """
    data = request.get_json(silent=True)
    if not data or not isinstance(data.get('vulnerabilities'), list):
        return jsonify({
            "status": "error",
            "message": "Request body must contain a 'vulnerabilities'
array",
            "code": 400
        }), 400
```

```
    items = data['vulnerabilities']

    if len(items) < 2:
        return jsonify({
            "status": "error",
            "message": "Bulk create requires at least 2 items",
            "code": 400
        }), 400

    if len(items) > 50:
        return jsonify({
            "status": "error",
            "message": "Bulk create is limited to 50 items per request",
            "code": 400
        }), 400

    # Validate every item first — fail-fast on the whole batch
    item_errors = {}
    for idx, item in enumerate(items):
        if not isinstance(item, dict):
            item_errors[idx] = "Item must be a JSON object"
            continue
        is_valid, error_msg = validate_vulnerability_input(item,
is_update=False)
        if not is_valid:
            item_errors[idx] = error_msg

    if item_errors:
        return jsonify({
            "status": "error",
            "message": "Validation failed for one or more items",
            "errors": {str(k): v for k, v in item_errors.items()},
            "code": 422
        }), 422

    now = utcnow()
    docs = []
    for item in items:
        docs.append({
            "vulnerability_title": item['vulnerability_title'].strip(),
            "description": item['description'].strip(),
            "cve_id": item.get('cve_id', '').strip() or None,
            "severity": item['severity'],
            "status": item['status'],
            "cvss_score": float(item['cvss_score']),
            "asset_name": item['asset_name'].strip(),
            "asset_type": item['asset_type'],
            "vulnerability_type": item['vulnerability_type'],
            "discovery_method": item['discovery_method'],
            "department": item['department'].strip(),
            "affected_versions": item.get('affected_versions', []),
            "attack_vector": item.get('attack_vector'),
            "exploitability": item.get('exploitability'),
```

```
            "patch_due_date": parse_iso_date(item.get('patch_due_date')),
            "patch_applied": item.get('patch_applied', False),
            "assigned_to": item.get('assigned_to', '').strip() or None,
            "reported_by": item['reported_by'].strip(),
            "risk_score": calculate_risk_score(
                float(item['cvss_score']),
                item.get('exploitability'),
                item['severity'],
            ),
            "remediation_steps": [],
            "activity_log": [],
            "location": _build_geojson(item.get('location')),
            "created_at": now,
            "updated_at": now,
            "created_by": current_user['username'],
        })

    db = get_db()
    result = db.vulnerabilities.insert_many(docs)

    # Attach inserted IDs back to docs
    for doc, oid in zip(docs, result.inserted_ids):
        doc['_id'] = oid

    return jsonify({
        "status": "success",
        "data": [serialize_doc(d) for d in docs],
        "message": f"{len(docs)} vulnerabilities created successfully"
    }), 201


@vulnerabilities_bp.route('/<vuln_id>', methods=['PUT'])
@token_required
@role_required('admin', 'analyst')
def update_vulnerability(current_user: dict, vuln_id: str):
    """Update an existing vulnerability."""
    if not validate_object_id(vuln_id):
        return jsonify({"status": "error", "message": "Invalid ObjectId
format", "code": 422}), 422

    data = request.get_json(silent=True)
    if not data:
        return jsonify({"status": "error", "message": "Request body must
be valid JSON", "code": 400}), 400

    is_valid, error_msg = validate_vulnerability_input(data,
is_update=True)
    if not is_valid:
        return jsonify({"status": "error", "message": error_msg, "code":
422}), 422

    db = get_db()
    existing = db.vulnerabilities.find_one({"_id": ObjectId(vuln_id)})
    if not existing:
```

```
        return jsonify({"status": "error", "message": "Vulnerability not
found", "code": 404}), 404

    # Build update dict only with provided fields
    update_fields = {}
    string_fields = ['vulnerability_title', 'description', 'cve_id',
'asset_name', 'department',
                     'assigned_to', 'reported_by']
    for field in string_fields:
        if field in data:
            update_fields[field] = data[field].strip() if
isinstance(data[field], str) else data[field]

    enum_fields = ['severity', 'status', 'asset_type',
'vulnerability_type',
                   'discovery_method', 'attack_vector', 'exploitability']
    for field in enum_fields:
        if field in data:
            update_fields[field] = data[field]

    if 'cvss_score' in data:
        update_fields['cvss_score'] = float(data['cvss_score'])

    if 'patch_applied' in data:
        update_fields['patch_applied'] = data['patch_applied']

    if 'affected_versions' in data:
        update_fields['affected_versions'] = data['affected_versions']

    if 'patch_due_date' in data:
        update_fields['patch_due_date'] =
parse_iso_date(data['patch_due_date'])

    if 'location' in data:
        update_fields['location'] = _build_geojson(data.get('location'))

    update_fields['updated_at'] = utcnow()

    # Recalculate risk score if relevant fields changed
    cvss = update_fields.get('cvss_score', existing.get('cvss_score', 0))
    exploit = update_fields.get('exploitability',
existing.get('exploitability'))
    sev = update_fields.get('severity', existing.get('severity'))
    update_fields['risk_score'] = calculate_risk_score(cvss, exploit, sev)

    db.vulnerabilities.update_one(
        {"_id": ObjectId(vuln_id)},
        {"$set": update_fields}
    )

    updated_vuln = db.vulnerabilities.find_one({"_id": ObjectId(vuln_id)})
    return jsonify({
        "status": "success",
        "data": serialize_doc(updated_vuln),
```

```
        "message": "Vulnerability updated successfully"
    }), 200


@vulnerabilities_bp.route('/<vuln_id>', methods=['DELETE'])
@token_required
@role_required('admin')
def delete_vulnerability(current_user: dict, vuln_id: str):
    """Delete a vulnerability permanently."""
    if not validate_object_id(vuln_id):
        return jsonify({"status": "error", "message": "Invalid ObjectId
format", "code": 422}), 422

    db = get_db()
    result = db.vulnerabilities.delete_one({"_id": ObjectId(vuln_id)})

    if result.deleted_count == 0:
        return jsonify({"status": "error", "message": "Vulnerability not
found", "code": 404}), 404

    return '', 204


# ── Geospatial helpers & endpoint ──────────────────────────────────

def _build_geojson(loc: dict | None) -> dict | None:
    """Convert a location payload into a GeoJSON Point document.

    Accepts either ``{"type": "Point", "coordinates": [lng, lat]}``
    (raw GeoJSON) or a convenience object ``{"lng": ..., "lat": ...}``.

    Returns:
        GeoJSON Point dict or *None* if the input is invalid / absent.
    """
    if not loc or not isinstance(loc, dict):
        return None

    if loc.get('type') == 'Point' and isinstance(loc.get('coordinates'),
list):
        coords = loc['coordinates']
        if len(coords) >= 2 and all(isinstance(c, (int, float)) for c in
coords[:2]):
            return {"type": "Point", "coordinates": [float(coords[0]),
float(coords[1])]}

    if 'lng' in loc and 'lat' in loc:
        try:
            return {
                "type": "Point",
                "coordinates": [float(loc['lng']), float(loc['lat'])]
            }
        except (ValueError, TypeError):
            return None
```

```
    return None


@vulnerabilities_bp.route('/nearby', methods=['GET'])
def nearby_vulnerabilities():
    """Find vulnerabilities with assets near a geographic point.

    Query parameters:
        lng (float): Longitude of reference point.
        lat (float): Latitude of reference point.
        radius (float): Search radius in **kilometres** (default 50).
        limit (int): Maximum results (default 10, max 100).

    Uses MongoDB ``$geoNear`` aggregation stage with a ``2dsphere`` index.
    """
    args = request.args

    try:
        lng = float(args['lng'])
        lat = float(args['lat'])
    except (KeyError, ValueError, TypeError):
        return jsonify({
            "status": "error",
            "message": "'lng' and 'lat' query parameters are required and
must be numbers",
            "code": 400
        }), 400

    try:
        radius_km = float(args.get('radius', 50))
    except (ValueError, TypeError):
        radius_km = 50.0

    try:
        limit = min(int(args.get('limit', 10)), 100)
    except (ValueError, TypeError):
        limit = 10

    db = get_db()

    pipeline = [
        {"$geoNear": {
            "near": {"type": "Point", "coordinates": [lng, lat]},
            "distanceField": "distance_metres",
            "maxDistance": radius_km * 1000,  # convert km → m
            "spherical": True,
        }},
        {"$limit": limit},
        {"$project": {
            "vulnerability_title": 1,
            "severity": 1,
            "cvss_score": 1,
            "asset_name": 1,
            "department": 1,
```

```
            "location": 1,
            "distance_km": {"$round": [{"$divide": ["$distance_metres",
1000]}, 2]},
        }},
    ]

    result = list(db.vulnerabilities.aggregate(pipeline))
    return jsonify({
        "status": "success",
        "data": [serialize_doc(r) for r in result]
    }), 200
```

---

9. routes/vulnerabilities/validation.py

File: backend/routes/vulnerabilities/validation.py

```
"""Input validation for vulnerability endpoints."""

from utils.validators import (
    validate_string, validate_enum, validate_number, validate_date,
validate_cve_id,
    VALID_SEVERITIES, VALID_STATUSES, VALID_ASSET_TYPES,
VALID_ATTACK_VECTORS,
    VALID_EXPLOITABILITIES, VALID_VULNERABILITY_TYPES,
VALID_DISCOVERY_METHODS,
)


def validate_vulnerability_input(data: dict, is_update: bool = False) ->
tuple[bool, str]:
    """Validate vulnerability creation or update input.

    Validates all required fields (vulnerability_title, description,
severity, status,
    cvss_score, asset_name, asset_type, vulnerability_type,
discovery_method,
    department, reported_by) when creating. For updates, only validates
present fields.

    Optional fields validated when present: cve_id, attack_vector,
exploitability,
    patch_due_date, assigned_to, patch_applied, affected_versions array.
    """
    errors = []
    required = not is_update

    # Validates each field using utils.validators functions
    # Required fields: validate_string() for titles/names, validate_enum()
for status/severity,
    #                  validate_number() for cvss_score
```

```
    # Optional fields: validate_cve_id(), validate_date(), array type
checking
    # ... (full validation logic in source file)

    return (False, "; ".join(errors)) if errors else (True, "")
```

---

10. routes/remediation/__init__.py

File: backend/routes/remediation/__init__.py

```
"""Remediation blueprint — CRUD for remediation steps sub-documents."""

from flask import Blueprint

remediation_bp = Blueprint('remediation', __name__,
url_prefix='/api/v1/vulnerabilities')

from routes.remediation import routes  # noqa: E402, F401
```

---

11. routes/remediation/routes.py

File: backend/routes/remediation/routes.py

```
"""Remediation steps sub-document CRUD endpoints."""

from flask import request, jsonify
from bson import ObjectId

from middleware.auth_middleware import token_required
from middleware.rbac_middleware import role_required
from routes.remediation.validation import validate_remediation_step_input
from utils.helpers import get_db, validate_object_id, serialize_doc,
utcnow
from utils.validators import parse_iso_date
from routes.remediation import remediation_bp


@remediation_bp.route('/<vuln_id>/remediation-steps', methods=['GET'])
@token_required
def get_remediation_steps(current_user: dict, vuln_id: str):
    """Retrieve all remediation steps for a vulnerability."""
    if not validate_object_id(vuln_id):
        return jsonify({"status": "error", "message": "Invalid ObjectId
format", "code": 422}), 422

    db = get_db()
    vuln = db.vulnerabilities.find_one({"_id": ObjectId(vuln_id)})
```

```
    if not vuln:
        return jsonify({"status": "error", "message": "Vulnerability not
found", "code": 404}), 404

    steps = vuln.get('remediation_steps', [])
    return jsonify({
        "status": "success",
        "data": [serialize_doc(s) for s in steps]
    }), 200


@remediation_bp.route('/<vuln_id>/remediation-steps/<step_id>', methods=
['GET'])
@token_required
def get_remediation_step(current_user: dict, vuln_id: str, step_id: str):
    """Retrieve a single remediation step by ID."""
    if not validate_object_id(vuln_id) or not validate_object_id(step_id):
        return jsonify({"status": "error", "message": "Invalid ObjectId
format", "code": 422}), 422

    db = get_db()

    # Use aggregation to extract single sub-document
    pipeline = [
        {"$match": {"_id": ObjectId(vuln_id)}},
        {"$unwind": "$remediation_steps"},
        {"$match": {"remediation_steps._id": ObjectId(step_id)}},
        {"$replaceRoot": {"newRoot": "$remediation_steps"}},
    ]
    results = list(db.vulnerabilities.aggregate(pipeline))

    if not results:
        return jsonify({"status": "error", "message": "Remediation step
not found", "code": 404}), 404

    return jsonify({"status": "success", "data":
serialize_doc(results[0])}), 200


@remediation_bp.route('/<vuln_id>/remediation-steps', methods=['POST'])
@token_required
@role_required('admin', 'analyst')
def add_remediation_step(current_user: dict, vuln_id: str):
    """Add a new remediation step to a vulnerability."""
    if not validate_object_id(vuln_id):
        return jsonify({"status": "error", "message": "Invalid ObjectId
format", "code": 422}), 422

    data = request.get_json(silent=True)
    if not data:
        return jsonify({"status": "error", "message": "Request body must
be valid JSON", "code": 400}), 400
```

```
    is_valid, error_msg = validate_remediation_step_input(data,
is_update=False)
    if not is_valid:
        return jsonify({"status": "error", "message": error_msg, "code":
422}), 422

    db = get_db()
    vuln = db.vulnerabilities.find_one({"_id": ObjectId(vuln_id)})
    if not vuln:
        return jsonify({"status": "error", "message": "Vulnerability not
found", "code": 404}), 404

    new_step = {
        "_id": ObjectId(),
        "step_number": int(data['step_number']),
        "step_description": data['step_description'].strip(),
        "recommended_by": data.get('recommended_by', '').strip() or None,
        "status": data['status'],
        "due_date": parse_iso_date(data.get('due_date')),
        "completed_date": None,
        "notes": data.get('notes', '').strip() or None,
        "added_at": utcnow(),
    }

    db.vulnerabilities.update_one(
        {"_id": ObjectId(vuln_id)},
        {
            "$push": {"remediation_steps": new_step},
            "$set": {"updated_at": utcnow()},
        }
    )

    return jsonify({
        "status": "success",
        "data": serialize_doc(new_step),
        "message": "Remediation step added successfully"
    }), 201


@remediation_bp.route('/<vuln_id>/remediation-steps/<step_id>', methods=
['PUT'])
@token_required
@role_required('admin', 'analyst')
def update_remediation_step(current_user: dict, vuln_id: str, step_id:
str):
    """Update a specific remediation step by ID."""
    if not validate_object_id(vuln_id) or not validate_object_id(step_id):
        return jsonify({"status": "error", "message": "Invalid ObjectId
format", "code": 422}), 422

    data = request.get_json(silent=True)
    if not data:
        return jsonify({"status": "error", "message": "Request body must
be valid JSON", "code": 400}), 400
```

```
    is_valid, error_msg = validate_remediation_step_input(data,
is_update=True)
    if not is_valid:
        return jsonify({"status": "error", "message": error_msg, "code":
422}), 422

    db = get_db()

    # Check vulnerability and step exist
    vuln = db.vulnerabilities.find_one({
        "_id": ObjectId(vuln_id),
        "remediation_steps._id": ObjectId(step_id),
    })
    if not vuln:
        return jsonify({"status": "error", "message": "Remediation step
not found", "code": 404}), 404

    # Build positional update
    set_fields = {}
    if 'step_number' in data:
        set_fields['remediation_steps.$.step_number'] =
int(data['step_number'])
    if 'step_description' in data:
        set_fields['remediation_steps.$.step_description'] =
data['step_description'].strip()
    if 'recommended_by' in data:
        set_fields['remediation_steps.$.recommended_by'] =
data['recommended_by'].strip() if data['recommended_by'] else None
    if 'status' in data:
        set_fields['remediation_steps.$.status'] = data['status']
        if data['status'] == 'Completed':
            set_fields['remediation_steps.$.completed_date'] = utcnow()
    if 'due_date' in data:
        set_fields['remediation_steps.$.due_date'] =
parse_iso_date(data['due_date'])
    if 'notes' in data:
        set_fields['remediation_steps.$.notes'] = data['notes'].strip() if
data['notes'] else None

    set_fields['updated_at'] = utcnow()

    db.vulnerabilities.update_one(
        {"_id": ObjectId(vuln_id), "remediation_steps._id":
ObjectId(step_id)},
        {"$set": set_fields}
    )

    # Retrieve updated step
    pipeline = [
        {"$match": {"_id": ObjectId(vuln_id)}},
        {"$unwind": "$remediation_steps"},
        {"$match": {"remediation_steps._id": ObjectId(step_id)}},
        {"$replaceRoot": {"newRoot": "$remediation_steps"}},
```

```
    ]
    results = list(db.vulnerabilities.aggregate(pipeline))

    return jsonify({
        "status": "success",
        "data": serialize_doc(results[0]) if results else {},
        "message": "Remediation step updated successfully"
    }), 200


@remediation_bp.route('/<vuln_id>/remediation-steps/<step_id>', methods=
['DELETE'])
@token_required
@role_required('admin', 'analyst')
def delete_remediation_step(current_user: dict, vuln_id: str, step_id:
str):
    """Remove a remediation step from a vulnerability."""
    if not validate_object_id(vuln_id) or not validate_object_id(step_id):
        return jsonify({"status": "error", "message": "Invalid ObjectId
format", "code": 422}), 422

    db = get_db()

    result = db.vulnerabilities.update_one(
        {"_id": ObjectId(vuln_id)},
        {
            "$pull": {"remediation_steps": {"_id": ObjectId(step_id)}},
            "$set": {"updated_at": utcnow()},
        }
    )

    if result.matched_count == 0:
        return jsonify({"status": "error", "message": "Vulnerability not
found", "code": 404}), 404

    if result.modified_count == 0:
        return jsonify({"status": "error", "message": "Remediation step
not found", "code": 404}), 404

    return '', 204
```

---

12. routes/remediation/validation.py

File: `backend/routes/remediation/validation.py`

```
"""Input validation for remediation step endpoints."""

from utils.validators import (
    validate_string, validate_enum, validate_number, validate_date,
    VALID_REMEDIATION_STATUSES,
```

```
)


def validate_remediation_step_input(data: dict, is_update: bool = False) -
> tuple[bool, str]:
    """Validate remediation step input (step_number, step_description,
status required;
    optional: recommended_by, due_date, notes). Validates using
utils.validators."""
    errors = []
    required = not is_update
    # Validates step_number (positive integer), step_description (5-1000
chars),
    # status (enum), recommended_by, due_date (ISO 8601), notes
    # ... (full validation in source file)
    return (False, "; ".join(errors)) if errors else (True, "")
```

---

13. routes/activity_log/__init__.py

File: `backend/routes/activity_log/__init__.py`

```
"""Activity Log blueprint - CRUD for vulnerability activity log
entries."""

from flask import Blueprint

activity_log_bp = Blueprint('activity_log', __name__,
url_prefix='/api/v1/vulnerabilities')

from routes.activity_log import routes  # noqa: E402, F401
```

---

14. routes/activity_log/routes.py

File: `backend/routes/activity_log/routes.py`

```
"""Activity log sub-document CRUD endpoints."""

from flask import request, jsonify
from bson import ObjectId

from middleware.auth_middleware import token_required
from middleware.rbac_middleware import role_required
from routes.activity_log.validation import validate_activity_log_input
from utils.helpers import get_db, validate_object_id, serialize_doc,
utcnow
from routes.activity_log import activity_log_bp
```

```
@activity_log_bp.route('/<vuln_id>/activity-log', methods=['GET'])
@token_required
def get_activity_log(current_user: dict, vuln_id: str):
    """Retrieve all activity log entries for a vulnerability."""
    if not validate_object_id(vuln_id):
        return jsonify({"status": "error", "message": "Invalid ObjectId
format", "code": 422}), 422

    db = get_db()
    vuln = db.vulnerabilities.find_one({"_id": ObjectId(vuln_id)})

    if not vuln:
        return jsonify({"status": "error", "message": "Vulnerability not
found", "code": 404}), 404

    logs = vuln.get('activity_log', [])
    return jsonify({
        "status": "success",
        "data": [serialize_doc(log) for log in logs]
    }), 200


@activity_log_bp.route('/<vuln_id>/activity-log/<log_id>', methods=
['GET'])
@token_required
def get_activity_log_entry(current_user: dict, vuln_id: str, log_id: str):
    """Retrieve a single activity log entry by ID."""
    if not validate_object_id(vuln_id) or not validate_object_id(log_id):
        return jsonify({"status": "error", "message": "Invalid ObjectId
format", "code": 422}), 422

    db = get_db()

    pipeline = [
        {"$match": {"_id": ObjectId(vuln_id)}},
        {"$unwind": "$activity_log"},
        {"$match": {"activity_log._id": ObjectId(log_id)}},
        {"$replaceRoot": {"newRoot": "$activity_log"}},
    ]
    results = list(db.vulnerabilities.aggregate(pipeline))

    if not results:
        return jsonify({"status": "error", "message": "Activity log entry
not found", "code": 404}), 404

    return jsonify({"status": "success", "data":
serialize_doc(results[0])}), 200


@activity_log_bp.route('/<vuln_id>/activity-log', methods=['POST'])
@token_required
@role_required('admin', 'analyst')
def add_activity_log_entry(current_user: dict, vuln_id: str):
```

```
    """Add a new activity log entry to a vulnerability."""
    if not validate_object_id(vuln_id):
        return jsonify({"status": "error", "message": "Invalid ObjectId
format", "code": 422}), 422

    data = request.get_json(silent=True)
    if not data:
        return jsonify({"status": "error", "message": "Request body must
be valid JSON", "code": 400}), 400

    is_valid, error_msg = validate_activity_log_input(data)
    if not is_valid:
        return jsonify({"status": "error", "message": error_msg, "code":
422}), 422

    db = get_db()
    vuln = db.vulnerabilities.find_one({"_id": ObjectId(vuln_id)})
    if not vuln:
        return jsonify({"status": "error", "message": "Vulnerability not
found", "code": 404}), 404

    new_entry = {
        "_id": ObjectId(),
        "performed_at": utcnow(),
        "action": data['action'].strip(),
        "performed_by": current_user['username'],
        "details": data.get('details', '').strip() or None,
        "previous_value": data.get('previous_value', '').strip() or None,
        "new_value": data.get('new_value', '').strip() or None,
    }

    db.vulnerabilities.update_one(
        {"_id": ObjectId(vuln_id)},
        {
            "$push": {"activity_log": new_entry},
            "$set": {"updated_at": utcnow()},
        }
    )

    return jsonify({
        "status": "success",
        "data": serialize_doc(new_entry),
        "message": "Activity log entry added successfully"
    }), 201


@activity_log_bp.route('/<vuln_id>/activity-log/<log_id>', methods=
['DELETE'])
@token_required
@role_required('admin')
def delete_activity_log_entry(current_user: dict, vuln_id: str, log_id:
str):
    """Delete an activity log entry (Admin only)."""
    if not validate_object_id(vuln_id) or not validate_object_id(log_id):
```

```
        return jsonify({"status": "error", "message": "Invalid ObjectId
format", "code": 422}), 422

    db = get_db()

    result = db.vulnerabilities.update_one(
        {"_id": ObjectId(vuln_id)},
        {
            "$pull": {"activity_log": {"_id": ObjectId(log_id)}},
            "$set": {"updated_at": utcnow()},
        }
    )

    if result.matched_count == 0:
        return jsonify({"status": "error", "message": "Vulnerability not
found", "code": 404}), 404

    if result.modified_count == 0:
        return jsonify({"status": "error", "message": "Activity log entry
not found", "code": 404}), 404

    return '', 204
```

---

15. routes/activity_log/validation.py

File: `backend/routes/activity_log/validation.py`

```
"""Input validation for activity log endpoints."""

from utils.validators import validate_string


def validate_activity_log_input(data: dict) -> tuple[bool, str]:
    """Validate activity log entry input (action required; optional:
details,
    previous_value, new_value). Uses validate_string() with length
constraints."""
    errors = []
    # Validates action (required, 1-200 chars), details (0-2000),
    # previous_value (0-500), new_value (0-500)
    # ... (full validation in source file)
    return (False, "; ".join(errors)) if errors else (True, "")
```

---

16. routes/analytics/__init__.py

File: `backend/routes/analytics/__init__.py`

```python
"""Analytics blueprint — aggregation pipelines and dashboard KPIs."""

from flask import Blueprint

analytics_bp = Blueprint('analytics', __name__,
url_prefix='/api/v1/analytics')

from routes.analytics import routes  # noqa: E402, F401
```

---

17. routes/analytics/routes.py

File: backend/routes/analytics/routes.py

```python
"""Analytics aggregation pipeline routes."""

from datetime import datetime, timezone
from flask import request, jsonify

from middleware.auth_middleware import token_required
from middleware.rbac_middleware import role_required
from utils.helpers import get_db, serialize_doc, utcnow
from routes.analytics import analytics_bp


@analytics_bp.route('/severity-distribution', methods=['GET'])
@token_required
def severity_distribution(current_user: dict):
    """Count of vulnerabilities grouped by severity level."""
    db = get_db()

    pipeline = [
        {"$group": {
            "_id": "$severity",
            "count": {"$sum": 1},
            "avg_cvss": {"$avg": "$cvss_score"}
        }},
        {"$project": {
            "severity": "$_id",
            "count": 1,
            "avg_cvss": {"$round": ["$avg_cvss", 2]},
            "_id": 0
        }},
        {"$sort": {"count": -1}}
    ]

    result = list(db.vulnerabilities.aggregate(pipeline))
    return jsonify({"status": "success", "data": result}), 200
```

```
@analytics_bp.route('/department-risk', methods=['GET'])
@token_required
def department_risk(current_user: dict):
    """Risk exposure analysis grouped by department."""
    db = get_db()

    pipeline = [
        {"$group": {
            "_id": "$department",
            "total_vulnerabilities": {"$sum": 1},
            "critical_count": {
                "$sum": {"$cond": [{"$eq": ["$severity", "Critical"]}, 1,
0]}
            },
            "high_count": {
                "$sum": {"$cond": [{"$eq": ["$severity", "High"]}, 1, 0]}
            },
            "avg_cvss": {"$avg": "$cvss_score"},
            "max_cvss": {"$max": "$cvss_score"}
        }},
        {"$project": {
            "department": "$_id",
            "total_vulnerabilities": 1,
            "critical_count": 1,
            "high_count": 1,
            "avg_cvss": {"$round": ["$avg_cvss", 2]},
            "max_cvss": 1,
            "risk_score": {
                "$round": [{
                    "$add": [
                        {"$multiply": ["$critical_count", 4]},
                        {"$multiply": ["$high_count", 2]},
                        "$avg_cvss"
                    ]
                }, 2]
            },
            "_id": 0
        }},
        {"$sort": {"risk_score": -1}}
    ]

    result = list(db.vulnerabilities.aggregate(pipeline))
    return jsonify({"status": "success", "data": result}), 200


@analytics_bp.route('/overdue-patches', methods=['GET'])
@token_required
def overdue_patches(current_user: dict):
    """List of vulnerabilities with overdue patch deadlines."""
    db = get_db()
    args = request.args
    now = utcnow()

    match_stage = {
```

```
        "patch_applied": False,
        "patch_due_date": {"$lt": now},
        "status": {"$nin": ["Patched", "Accepted Risk"]}
    }

    if args.get('severity'):
        match_stage['severity'] = args['severity']
    if args.get('department'):
        match_stage['department'] = args['department']

    pipeline = [
        {"$match": match_stage},
        {"$project": {
            "vulnerability_title": 1,
            "severity": 1,
            "cvss_score": 1,
            "asset_name": 1,
            "department": 1,
            "patch_due_date": 1,
            "days_overdue": {
                "$dateDiff": {
                    "startDate": "$patch_due_date",
                    "endDate": now,
                    "unit": "day"
                }
            }
        }},
        {"$sort": {"days_overdue": -1}}
    ]

    result = list(db.vulnerabilities.aggregate(pipeline))
    return jsonify({"status": "success", "data": [serialize_doc(r) for r
in result]}), 200


@analytics_bp.route('/patch-compliance', methods=['GET'])
@token_required
def patch_compliance(current_user: dict):
    """Overall patch compliance rate statistics."""
    db = get_db()

    pipeline = [
        {"$group": {
            "_id": None,
            "total": {"$sum": 1},
            "patched": {
                "$sum": {"$cond": ["$patch_applied", 1, 0]}
            },
            "unpatched": {
                "$sum": {"$cond": ["$patch_applied", 0, 1]}
            }
        }},
        {"$project": {
            "_id": 0,
```

```
            "total": 1,
            "patched": 1,
            "unpatched": 1,
            "compliance_rate": {
                "$round": [
                    {"$cond": [
                        {"$eq": ["$total", 0]},
                        0,
                        {"$multiply": [
                            {"$divide": ["$patched", "$total"]},
                            100
                        ]}
                    ]},
                    2
                ]
            }
        }}
    ]

    result = list(db.vulnerabilities.aggregate(pipeline))
    data = result[0] if result else {"total": 0, "patched": 0,
"unpatched": 0, "compliance_rate": 0}
    return jsonify({"status": "success", "data": data}), 200


@analytics_bp.route('/vulnerability-trends', methods=['GET'])
@token_required
def vulnerability_trends(current_user: dict):
    """Vulnerability count trend grouped by month."""
    db = get_db()
    args = request.args

    try:
        months = int(args.get('months', 12))
    except (ValueError, TypeError):
        months = 12

    # Calculate the start date as N months ago
    now = utcnow()
    start_year = now.year
    start_month = now.month - months
    while start_month <= 0:
        start_month += 12
        start_year -= 1
    from datetime import datetime
    start_date = datetime(start_year, start_month, 1)

    pipeline = [
        {"$match": {"created_at": {"$gte": start_date}}},
        {"$group": {
            "_id": {
                "year": {"$year": "$created_at"},
                "month": {"$month": "$created_at"}
            },
```

```
            "count": {"$sum": 1}
        }},
        {"$sort": {"_id.year": 1, "_id.month": 1}},
        {"$project": {
            "_id": 0,
            "year": "$_id.year",
            "month": "$_id.month",
            "count": 1
        }}
    ]

    result = list(db.vulnerabilities.aggregate(pipeline))
    return jsonify({"status": "success", "data": result}), 200


@analytics_bp.route('/top-affected-assets', methods=['GET'])
@token_required
def top_affected_assets(current_user: dict):
    """Top N assets ranked by vulnerability count."""
    db = get_db()
    args = request.args

    try:
        limit = min(int(args.get('limit', 10)), 100)
    except (ValueError, TypeError):
        limit = 10

    pipeline = [
        {"$group": {
            "_id": {
                "asset_name": "$asset_name",
                "asset_type": "$asset_type"
            },
            "count": {"$sum": 1},
            "avg_cvss": {"$avg": "$cvss_score"}
        }},
        {"$project": {
            "_id": 0,
            "asset_name": "$_id.asset_name",
            "asset_type": "$_id.asset_type",
            "count": 1,
            "avg_cvss": {"$round": ["$avg_cvss", 2]}
        }},
        {"$sort": {"count": -1}},
        {"$limit": limit}
    ]

    result = list(db.vulnerabilities.aggregate(pipeline))
    return jsonify({"status": "success", "data": result}), 200


@analytics_bp.route('/mean-time-to-remediation', methods=['GET'])
@token_required
def mean_time_to_remediation(current_user: dict):
```

```
    """Average days to resolve vulnerabilities by severity."""
    db = get_db()

    pipeline = [
        {"$match": {
            "status": {"$in": ["Patched", "Accepted Risk"]},
            "updated_at": {"$exists": True}
        }},
        {"$project": {
            "severity": 1,
            "resolution_days": {
                "$dateDiff": {
                    "startDate": "$created_at",
                    "endDate": "$updated_at",
                    "unit": "day"
                }
            }
        }},
        {"$group": {
            "_id": "$severity",
            "avg_days": {"$avg": "$resolution_days"},
            "min_days": {"$min": "$resolution_days"},
            "max_days": {"$max": "$resolution_days"},
            "count": {"$sum": 1}
        }},
        {"$project": {
            "severity": "$_id",
            "avg_days": {"$round": ["$avg_days", 1]},
            "min_days": 1,
            "max_days": 1,
            "count": 1,
            "_id": 0
        }},
        {"$sort": {"avg_days": -1}}
    ]

    result = list(db.vulnerabilities.aggregate(pipeline))
    return jsonify({"status": "success", "data": result}), 200


@analytics_bp.route('/risk-scores', methods=['GET'])
@token_required
@role_required('admin', 'analyst')
def risk_scores(current_user: dict):
    """Calculated risk scores per vulnerability."""
    db = get_db()
    args = request.args

    match_stage = {}
    if args.get('min_score'):
        try:
            match_stage['risk_score'] = {"$gte": float(args['min_score'])}
        except ValueError:
            pass
```

```
    if args.get('department'):
        match_stage['department'] = args['department']

    pipeline = [
        {"$match": match_stage} if match_stage else {"$match": {}},
        {"$project": {
            "vulnerability_title": 1,
            "severity": 1,
            "cvss_score": 1,
            "exploitability": 1,
            "asset_name": 1,
            "department": 1,
            "risk_score": 1,
            "status": 1
        }},
        {"$sort": {"risk_score": -1}}
    ]

    result = list(db.vulnerabilities.aggregate(pipeline))
    return jsonify({"status": "success", "data": [serialize_doc(r) for r
in result]}), 200


@analytics_bp.route('/summary', methods=['GET'])
@token_required
def dashboard_summary(current_user: dict):
    """Dashboard summary KPIs (counts, rates, averages)."""
    db = get_db()

    pipeline = [
        {"$facet": {
            "status_counts": [
                {"$group": {
                    "_id": "$status",
                    "count": {"$sum": 1}
                }}
            ],
            "severity_counts": [
                {"$group": {
                    "_id": "$severity",
                    "count": {"$sum": 1}
                }}
            ],
            "totals": [
                {"$group": {
                    "_id": None,
                    "total_vulnerabilities": {"$sum": 1},
                    "avg_cvss": {"$avg": "$cvss_score"},
                    "patched": {
                        "$sum": {"$cond": ["$patch_applied", 1, 0]}
                    },
                    "total_with_deadline": {
                        "$sum": {"$cond": [
                            {"$and": [
```

```
                                {"$ne": ["$patch_due_date", None]},
                                {"$eq": ["$patch_applied", False]},
                                {"$lt": ["$patch_due_date", utcnow()]}
                            ]},
                            1, 0
                        ]}
                    }
                }}
            ]
        }}
    ]

    result = list(db.vulnerabilities.aggregate(pipeline))

    if not result:
        data = {
            "total_vulnerabilities": 0,
            "open_count": 0,
            "in_progress_count": 0,
            "resolved_count": 0,
            "closed_count": 0,
            "critical_count": 0,
            "high_count": 0,
            "avg_cvss": 0,
            "compliance_rate": 0,
            "overdue_count": 0,
        }
    else:
        facets = result[0]

        # Parse status counts
        status_map = {item['_id']: item['count'] for item in
facets.get('status_counts', [])}
        severity_map = {item['_id']: item['count'] for item in
facets.get('severity_counts', [])}
        totals = facets.get('totals', [{}])[0] if facets.get('totals')
else {}

        total = totals.get('total_vulnerabilities', 0)
        patched = totals.get('patched', 0)
        compliance_rate = round((patched / total * 100), 2) if total > 0
else 0

        data = {
            "total_vulnerabilities": total,
            "open_count": status_map.get('Open', 0),
            "in_progress_count": status_map.get('In Progress', 0),
            "resolved_count": status_map.get('Patched', 0),
            "closed_count": status_map.get('Accepted Risk', 0),
            "critical_count": severity_map.get('Critical', 0),
            "high_count": severity_map.get('High', 0),
            "avg_cvss": round(totals.get('avg_cvss', 0) or 0, 2),
            "compliance_rate": compliance_rate,
            "overdue_count": totals.get('total_with_deadline', 0),
```

```
        }

    return jsonify({"status": "success", "data": data}), 200


@analytics_bp.route('/generate-report', methods=['POST'])
@token_required
@role_required('admin', 'analyst')
def generate_report(current_user: dict):
    """Run a comprehensive aggregation pipeline and persist the results
    to a ``reports`` collection using the ``$out`` stage.

    The pipeline groups vulnerabilities by department and severity,
    calculates summary statistics, and writes the output as a snapshot
    report that can be retrieved later.
    """
    db = get_db()

    pipeline = [
        {"$group": {
            "_id": {
                "department": "$department",
                "severity": "$severity"
            },
            "count": {"$sum": 1},
            "avg_cvss": {"$avg": "$cvss_score"},
            "max_cvss": {"$max": "$cvss_score"},
            "patched": {"$sum": {"$cond": ["$patch_applied", 1, 0]}},
            "unpatched": {"$sum": {"$cond": ["$patch_applied", 0, 1]}},
        }},
        {"$project": {
            "_id": 0,
            "department": "$_id.department",
            "severity": "$_id.severity",
            "count": 1,
            "avg_cvss": {"$round": ["$avg_cvss", 2]},
            "max_cvss": 1,
            "patched": 1,
            "unpatched": 1,
        }},
        {"$sort": {"department": 1, "severity": 1}},
        {"$out": "reports"},
    ]

    db.vulnerabilities.aggregate(pipeline)

    # Read back the generated report collection
    report_docs = list(db.reports.find({}))

    return jsonify({
        "status": "success",
        "data": [serialize_doc(d) for d in report_docs],
        "message": f"Report generated — {len(report_docs)} rows written to
'reports' collection"
```

```
    }), 201


@analytics_bp.route('/reports', methods=['GET'])
@token_required
def get_reports(current_user: dict):
    """Retrieve the latest generated report from the ``reports``
collection."""
    db = get_db()
    docs = list(db.reports.find({}))

    if not docs:
        return jsonify({
            "status": "success",
            "data": [],
            "message": "No reports generated yet. POST
/analytics/generate-report first."
        }), 200

    return jsonify({
        "status": "success",
        "data": [serialize_doc(d) for d in docs]
    }), 200
```

---

18. routes/admin/__init__.py

File: backend/routes/admin/__init__.py

```
"""Admin blueprint — user management and administrative operations."""

from flask import Blueprint

admin_bp = Blueprint('admin', __name__, url_prefix='/api/v1/admin')

from routes.admin import routes  # noqa: E402, F401
```

---

19. routes/admin/routes.py

File: backend/routes/admin/routes.py

```
"""Admin management routes."""

import math
from flask import request, jsonify
from bson import ObjectId

from middleware.auth_middleware import token_required
```

```python
from middleware.rbac_middleware import role_required
from utils.helpers import get_db, validate_object_id, serialize_doc,
get_pagination_params
from utils.validators import VALID_ROLES
from routes.admin import admin_bp


@admin_bp.route('/users', methods=['GET'])
@token_required
@role_required('admin')
def list_users(current_user: dict):
    """List all registered users (Admin only)."""
    db = get_db()
    args = request.args

    page, per_page = get_pagination_params(args)

    query = {}
    if args.get('role'):
        query['role'] = args['role']
    if args.get('is_active') is not None and args.get('is_active') != '':
        query['is_active'] = args['is_active'].lower() == 'true'

    skip = (page - 1) * per_page
    total = db.users.count_documents(query)
    total_pages = math.ceil(total / per_page) if total > 0 else 1

    cursor = db.users.find(query, {"password_hash": 0}).sort("created_at",
-1).skip(skip).limit(per_page)
    users = [serialize_doc(u) for u in cursor]

    return jsonify({
        "status": "success",
        "data": users,
        "pagination": {
            "page": page,
            "per_page": per_page,
            "total": total,
            "pages": total_pages,
        }
    }), 200


@admin_bp.route('/users/<user_id>', methods=['GET'])
@token_required
@role_required('admin')
def get_user(current_user: dict, user_id: str):
    """Get a specific user's details (Admin only)."""
    if not validate_object_id(user_id):
        return jsonify({"status": "error", "message": "Invalid ObjectId
format", "code": 422}), 422

    db = get_db()
    user = db.users.find_one({"_id": ObjectId(user_id)}, {"password_hash":
```

```
0})

    if not user:
        return jsonify({"status": "error", "message": "User not found",
"code": 404}), 404

    return jsonify({"status": "success", "data": serialize_doc(user)}),
200


@admin_bp.route('/users/<user_id>/role', methods=['PUT'])
@token_required
@role_required('admin')
def update_user_role(current_user: dict, user_id: str):
    """Update a user's role (Admin only)."""
    if not validate_object_id(user_id):
        return jsonify({"status": "error", "message": "Invalid ObjectId
format", "code": 422}), 422

    data = request.get_json(silent=True)
    if not data:
        return jsonify({"status": "error", "message": "Request body must
be valid JSON", "code": 400}), 400

    new_role = data.get('role')
    if new_role not in VALID_ROLES:
        return jsonify({
            "status": "error",
            "message": f"'role' must be one of: {', '.join(VALID_ROLES)}",
            "code": 422
        }), 422

    db = get_db()

    # Prevent self-demotion
    if user_id == current_user['user_id']:
        return jsonify({"status": "error", "message": "Cannot change your
own role", "code": 400}), 400

    result = db.users.update_one({"_id": ObjectId(user_id)}, {"$set":
{"role": new_role}})

    if result.matched_count == 0:
        return jsonify({"status": "error", "message": "User not found",
"code": 404}), 404

    user = db.users.find_one({"_id": ObjectId(user_id)}, {"password_hash":
0})
    return jsonify({
        "status": "success",
        "data": serialize_doc(user),
        "message": f"User role updated to '{new_role}'"
    }), 200
```

```python
@admin_bp.route('/users/<user_id>/status', methods=['PUT'])
@token_required
@role_required('admin')
def update_user_status(current_user: dict, user_id: str):
    """Activate or deactivate a user account (Admin only)."""
    if not validate_object_id(user_id):
        return jsonify({"status": "error", "message": "Invalid ObjectId
format", "code": 422}), 422

    data = request.get_json(silent=True)
    if not data:
        return jsonify({"status": "error", "message": "Request body must
be valid JSON", "code": 400}), 400

    if 'is_active' not in data or not isinstance(data['is_active'], bool):
        return jsonify({
            "status": "error",
            "message": "'is_active' must be a boolean",
            "code": 422
        }), 422

    db = get_db()

    # Prevent self-deactivation
    if user_id == current_user['user_id']:
        return jsonify({"status": "error", "message": "Cannot deactivate
your own account", "code": 400}), 400

    result = db.users.update_one({"_id": ObjectId(user_id)}, {"$set":
{"is_active": data['is_active']}})

    if result.matched_count == 0:
        return jsonify({"status": "error", "message": "User not found",
"code": 404}), 404

    user = db.users.find_one({"_id": ObjectId(user_id)}, {"password_hash":
0})
    status_text = "activated" if data['is_active'] else "deactivated"
    return jsonify({
        "status": "success",
        "data": serialize_doc(user),
        "message": f"User account {status_text}"
    }), 200


@admin_bp.route('/users/<user_id>', methods=['DELETE'])
@token_required
@role_required('admin')
def delete_user(current_user: dict, user_id: str):
    """Permanently delete a user account (Admin only)."""
    if not validate_object_id(user_id):
        return jsonify({"status": "error", "message": "Invalid ObjectId
format", "code": 422}), 422
```

```
    db = get_db()

    # Prevent self-deletion
    if user_id == current_user['user_id']:
        return jsonify({"status": "error", "message": "Cannot delete your
own account", "code": 400}), 400

    result = db.users.delete_one({"_id": ObjectId(user_id)})

    if result.deleted_count == 0:
        return jsonify({"status": "error", "message": "User not found",
"code": 404}), 404

    return '', 204
```

---

20. middleware/auth_middleware.py

File: `backend/middleware/auth_middleware.py`

```
"""JWT authentication middleware decorator."""

from functools import wraps
from flask import request, jsonify, current_app
import jwt


def _extract_token() -> str | None:
    """Extract JWT from the request.

    Reads the token from the ``x-access-token`` header.

    Returns:
        The raw JWT string, or *None* if not present.
    """
    return request.headers.get('x-access-token')


def token_required(f):
    """Decorator to enforce JWT authentication on a route.

    Extracts and validates the JWT token from the ``x-access-token``
    header.  The decoded token payload is passed as ``current_user``
    to the wrapped function.

    Additionally checks the ``blacklist`` collection — if the presented
    token has been revoked (via logout) the request is rejected.
    """
    @wraps(f)
    def decorated(*args, **kwargs):
```

```
        token = _extract_token()

        if not token:
            return jsonify({
                "status": "error",
                "message": "Authentication token is missing",
                "code": 401
            }), 401

        # Check token blacklist
        try:
            db = current_app.extensions["mongo_db"]
            if db.blacklist.find_one({"token": token}):
                return jsonify({
                    "status": "error",
                    "message": "Token has been cancelled",
                    "code": 401
                }), 401
        except Exception:
            # If the blacklist check fails we still reject gracefully
            pass

        try:
            payload = jwt.decode(
                token,
                current_app.config['SECRET_KEY'],
                algorithms=["HS256"]
            )
            current_user = payload
        except jwt.ExpiredSignatureError:
            return jsonify({
                "status": "error",
                "message": "Token has expired",
                "code": 401
            }), 401
        except jwt.InvalidTokenError:
            return jsonify({
                "status": "error",
                "message": "Invalid token",
                "code": 401
            }), 401

        return f(current_user, *args, **kwargs)
    return decorated
```

---

21. middleware/rbac_middleware.py

File: `backend/middleware/rbac_middleware.py`

```python
"""Role-based access control middleware decorator."""

from functools import wraps
from flask import jsonify


def role_required(*roles: str):
    """Decorator to enforce role-based access control.

    Args:
        *roles: Allowed role names (e.g., 'admin', 'analyst').

    Returns:
        Decorated function that checks user role before execution.
    """
    def decorator(f):
        @wraps(f)
        def decorated(current_user: dict, *args, **kwargs):
            if current_user.get('role') not in roles:
                return jsonify({
                    "status": "error",
                    "message": "You do not have permission to access this
resource",
                    "code": 403
                }), 403
            return f(current_user, *args, **kwargs)
        return decorated
    return decorator
```

---

22. middleware/error_handler.py

File: `backend/middleware/error_handler.py`

```python
"""Centralised error handler for the Flask application."""

from flask import Flask, jsonify


def register_error_handlers(app: Flask) -> None:
    """Register all error handlers with the Flask app.

    Args:
        app: Flask application instance.
    """

    @app.errorhandler(400)
    def bad_request(e):
        return jsonify({
            "status": "error",
```

```
            "message": str(e.description) if hasattr(e, 'description')
else "Bad request",
            "code": 400
        }), 400

    @app.errorhandler(401)
    def unauthorized(e):
        return jsonify({
            "status": "error",
            "message": "Unauthorized",
            "code": 401
        }), 401

    @app.errorhandler(403)
    def forbidden(e):
        return jsonify({
            "status": "error",
            "message": "Forbidden",
            "code": 403
        }), 403

    @app.errorhandler(404)
    def not_found(e):
        return jsonify({
            "status": "error",
            "message": "Resource not found",
            "code": 404
        }), 404

    @app.errorhandler(405)
    def method_not_allowed(e):
        return jsonify({
            "status": "error",
            "message": "Method not allowed",
            "code": 405
        }), 405

    @app.errorhandler(413)
    def request_entity_too_large(e):
        return jsonify({
            "status": "error",
            "message": "Request body too large",
            "code": 413
        }), 413

    @app.errorhandler(422)
    def unprocessable(e):
        return jsonify({
            "status": "error",
            "message": str(e.description) if hasattr(e, 'description')
else "Unprocessable entity",
            "code": 422
        }), 422
```

```
    @app.errorhandler(429)
    def too_many_requests(e):
        return jsonify({
            "status": "error",
            "message": "Too many requests. Please try again later.",
            "code": 429
        }), 429

    @app.errorhandler(500)
    def internal_error(e):
        return jsonify({
            "status": "error",
            "message": "An internal server error occurred",
            "code": 500
        }), 500

    @app.errorhandler(Exception)
    def handle_unexpected_error(e):
        app.logger.error(f"Unexpected error: {str(e)}")
        return jsonify({
            "status": "error",
            "message": "An unexpected error occurred",
            "code": 500
        }), 500
```

---

23. utils/helpers.py

File: `backend/utils/helpers.py`

```
"""Utility helper functions for the VulnGuard application."""

from __future__ import annotations

from datetime import datetime, timezone
from typing import TYPE_CHECKING

from bson import ObjectId
from bson.errors import InvalidId

if TYPE_CHECKING:
    from pymongo.database import Database

from flask import current_app


def get_db() -> "Database[dict]":
    """Return the MongoDB database instance attached to the current Flask
app.

    This helper exists so that Pylance / Pyright can resolve the type of
```

```
    ``current_app.db`` without a ``reportAttributeAccessIssue`` warning.

    Returns:
        The PyMongo ``Database`` object.
    """
    return current_app.extensions["mongo_db"]


def validate_object_id(id_string: str) -> bool:
    """Validate that a string is a valid MongoDB ObjectId.

    Args:
        id_string: String to validate.

    Returns:
        True if valid ObjectId, False otherwise.
    """
    try:
        ObjectId(id_string)
        return True
    except (InvalidId, TypeError):
        return False


def serialize_doc(doc: dict | None) -> dict:
    """Convert a MongoDB document to a JSON-serializable dict.

    Converts ObjectId fields to strings and datetime fields to ISO format.

    Args:
        doc: MongoDB document dictionary, or None.

    Returns:
        JSON-serializable dictionary (empty dict if *doc* is None).
    """
    if doc is None:
        return {}

    result = {}
    for key, value in doc.items():
        if isinstance(value, ObjectId):
            result[key] = str(value)
        elif isinstance(value, datetime):
            result[key] = value.isoformat() + 'Z' if value.tzinfo is None
else value.isoformat()
        elif isinstance(value, list):
            result[key] = [
                serialize_doc(item) if isinstance(item, dict) else
                str(item) if isinstance(item, ObjectId) else
                item.isoformat() + 'Z' if isinstance(item, datetime) and
item.tzinfo is None else
                item.isoformat() if isinstance(item, datetime) else
                item
                for item in value
```

```
            ]
        elif isinstance(value, dict):
            result[key] = serialize_doc(value)
        else:
            result[key] = value
    return result


def get_pagination_params(args: dict, default_per_page: int = 10,
max_per_page: int = 100) -> tuple[int, int]:
    """Extract and validate pagination parameters from request args.

    Args:
        args: Request query parameters.
        default_per_page: Default items per page.
        max_per_page: Maximum allowed items per page.

    Returns:
        Tuple of (page, per_page).
    """
    try:
        page = max(1, int(args.get('page', 1)))
    except (ValueError, TypeError):
        page = 1

    try:
        per_page = min(max(1, int(args.get('per_page',
default_per_page))), max_per_page)
    except (ValueError, TypeError):
        per_page = default_per_page

    return page, per_page


def utcnow() -> datetime:
    """Return the current UTC datetime (timezone-naive for MongoDB
compatibility).

    Returns:
        Current UTC datetime.
    """
    return datetime.now(timezone.utc).replace(tzinfo=None)


def calculate_risk_score(cvss_score: float, exploitability: str | None =
None, severity: str | None = None) -> float:
    """Calculate a weighted risk score from CVSS, exploitability and
severity.

    Args:
        cvss_score: CVSS score (0-10).
        exploitability: Exploitability level string.
        severity: Severity level string.
```

```
    Returns:
        Calculated risk score rounded to 2 decimal places.
    """
    exploit_weights = {
        'Unproven': 1.0,
        'Proof-of-Concept': 2.0,
        'Functional': 3.0,
        'High': 4.0,
    }
    severity_weights = {
        'Informational': 0.5,
        'Low': 1.0,
        'Medium': 2.0,
        'High': 3.0,
        'Critical': 4.0,
    }

    exploit_factor = exploit_weights.get(exploitability, 1.0) if
exploitability else 1.0
    severity_factor = severity_weights.get(severity, 1.0) if severity else
1.0

    risk_score = cvss_score * exploit_factor * severity_factor / 4.0
    return round(risk_score, 2)
```

---

24. utils/validators.py

File: `backend/utils/validators.py`

```
"""Shared validation logic for the VulnGuard application."""

import re
from datetime import datetime


# Enum constants for validation
VALID_SEVERITIES = ['Critical', 'High', 'Medium', 'Low', 'Informational']
VALID_STATUSES = ['Open', 'In Progress', 'Patched', 'Accepted Risk']
VALID_ASSET_TYPES = ['Server', 'Workstation', 'Network Device',
'Application',
                     'Database', 'Cloud Service', 'IoT Device',
'Endpoint']
VALID_VULNERABILITY_TYPES = ['Software', 'Configuration', 'Access
Control']
VALID_DISCOVERY_METHODS = ['Scan', 'Audit', 'Manual']
VALID_ATTACK_VECTORS = ['Network', 'Adjacent', 'Local', 'Physical']
VALID_EXPLOITABILITIES = ['Unproven', 'Proof-of-Concept', 'Functional',
'High']
VALID_REMEDIATION_STATUSES = ['Pending', 'In Progress', 'Completed',
'Skipped']
```

```python
VALID_ROLES = ['admin', 'analyst', 'guest']

# Regex patterns
CVE_ID_PATTERN = re.compile(r'^CVE-\d{4}-\d{4,}$')
EMAIL_PATTERN = re.compile(r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]
{2,}$')
PASSWORD_PATTERN = re.compile(r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*
[@$!%*?&#]).{8,}$')


def validate_string(value, field_name: str, min_len: int = 1, max_len: int
= 200,
                    required: bool = True) -> list[str]:
    """Validate string field: checks required, type, and length
constraints.
    Returns list of error messages (empty if valid)."""
    # Implementation checks None/empty, isinstance(str), length range
    # ...


def validate_enum(value, field_name: str, valid_values: list[str],
                  required: bool = True) -> list[str]:
    """Validate enum field: checks value is in allowed list."""
    # Implementation checks required and membership in valid_values
    # ...


def validate_number(value, field_name: str, min_val: float | None = None,
                    max_val: float | None = None, required: bool = True) -
> list[str]:
    """Validate numeric field: checks required, type, and range
constraints."""
    # Implementation checks None, isinstance(int,float), min/max bounds
    # ...


def validate_date(value, field_name: str, required: bool = False) ->
list[str]:
    """Validate ISO 8601 date string using datetime.fromisoformat()."""
    # Implementation parses value.replace('Z', '+00:00') and catches
ValueError
    # ...


def validate_cve_id(value) -> list[str]:
    """Validate CVE ID format: CVE-YYYY-NNNNN using CVE_ID_PATTERN
regex."""
    # ...


def validate_email(value: str) -> list[str]:
    """Validate email format using EMAIL_PATTERN regex."""
    # ...
```

```
def validate_password(value: str) -> list[str]:
    """Validate password strength: min 8 chars, uppercase, lowercase,
digit, special.
    Uses PASSWORD_PATTERN regex."""
    # ...


def parse_iso_date(value: str) -> datetime | None:
    """Parse ISO 8601 date string to datetime object (timezone-naive).
    Returns None if invalid."""
    # Implementation: datetime.fromisoformat(value.replace('Z',
'+00:00')).replace(tzinfo=None)
    # ...
```

---

25. seeds/seed_data.py

File: `backend/seeds/seed_data.py`

```
"""Database seeding script with comprehensive vulnerability data.

Usage:
    python -m seeds.seed_data [--reset|--reseed]

Populates the VulnGuard database with realistic vulnerability data
including
users, vulnerabilities with sub-documents (remediation steps and activity
logs).
"""

import os
import sys
import random
from datetime import datetime, timedelta, timezone
from bson import ObjectId
from pymongo import MongoClient
import bcrypt

sys.path.insert(0,
os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from config import DevelopmentConfig
from utils.helpers import calculate_risk_score


# Helper functions for date generation
def utcnow() -> datetime:
    """Return current UTC datetime (timezone-naive)."""
    return datetime.now(timezone.utc).replace(tzinfo=None)
```

```python
def random_date(start_days_ago: int = 180, end_days_ago: int = 0) ->
datetime:
    """Generate a random datetime within a range of days ago."""
    days = random.randint(end_days_ago, start_days_ago)
    return utcnow() - timedelta(days=days, hours=random.randint(0, 23),
                                minutes=random.randint(0, 59))


# Template data arrays (samples shown, full arrays in source file)
DEPARTMENTS = ["Engineering", "Finance", "Operations", "IT", "HR", ...]  #
10 items
ASSET_NAMES = ["web-app-01.example.com", "api-gateway-01.internal", ...]
# 26 items
ASSET_TYPES = ["Server", "Workstation", "Network Device", "Application",
...]  # 7 items
ASSET_LOCATIONS = [
    {"type": "Point", "coordinates": [-5.9301, 54.5973]},  # Belfast
    # ... 9 more GeoJSON points for cities worldwide
]
SEVERITIES = ["Critical", "High", "Medium", "Low", "Informational"]
STATUSES = ["Open", "In Progress", "Patched", "Accepted Risk"]
ATTACK_VECTORS = ["Network", "Adjacent", "Local", "Physical"]
VULNERABILITY_TEMPLATES = [...]  # 30 templates with CVE IDs, CVSS scores,
descriptions
REMEDIATION_ACTIONS = [...]  # 20 remediation action descriptions
ACTIVITY_ACTIONS = [...]  # 10 activity log action types


def create_users(db) -> list[dict]:
    """Create test user accounts (admin, analyst, guest) with bcrypt
passwords."""
    db.users.delete_many({})
    # Creates 3 user accounts with hashed passwords and different roles
    # ...


def generate_remediation_steps(created_at: datetime) -> list[dict]:
    """Generate 0-4 random remediation step sub-documents."""
    # Returns list of remediation step dicts with status, description,
dates
    # ...


def generate_activity_log(created_at: datetime, username: str) ->
list[dict]:
    """Generate activity log sub-documents (creation entry + 0-5
updates)."""
    # Returns list of activity log entry dicts with actions, timestamps,
users
    # ...


def create_vulnerabilities(db) -> int:
    """Create 110+ vulnerability documents with sub-documents from
```

```
templates."""
    # Generates variations of vulnerabilities across assets, departments,
locations
    # Each vulnerability includes remediation_steps and activity_log
arrays
    # ...


def create_indexes(db) -> None:
    """Create MongoDB indexes (single-field, compound, text, 2dsphere,
TTL)."""
    # Creates indexes for efficient querying on severity, status, dates,
    # text search, geospatial queries, and TTL for blacklist tokens
    # ...


def seed_database():
    """Main seeding function: connect, create users/vulnerabilities, build
indexes."""
    client = MongoClient(DevelopmentConfig.MONGO_URI)
    db = client.get_database()
    users = create_users(db)
    vuln_count = create_vulnerabilities(db)
    create_indexes(db)
    print(f"Seeded {vuln_count} vulnerabilities and {len(users)} users")


def reset_database():
    """Drop entire database for clean slate."""
    # ...


def reseed_database():
    """Full pipeline: drop database → seed data → create indexes."""
    # ...


if __name__ == '__main__':
    import argparse
    parser = argparse.ArgumentParser(description="VulnGuard Database
Seeder")
    group = parser.add_mutually_exclusive_group()
    group.add_argument('--reset', action='store_true', help='Drop
database')
    group.add_argument('--reseed', action='store_true', help='Drop then
re-seed')
    args = parser.parse_args()

    if args.reset:
        reset_database()
    elif args.reseed:
        reseed_database()
    else:
        seed_database()
```

> **Note:** The full `seed_data.py` is 796 lines with extensive constant arrays and generation logic. The above shows the structure; complete source is in the submitted code.

End of Code Listing